

Custom MapReduce with Hadoop Streaming for Online Food Delivery Analytics

Project Objective

Analyze online food delivery transactions using Hadoop Streaming to:

- Find city-wise revenue
- Identify top-selling food items
- Analyze monthly revenue trends
- Determine average order values
- Visualize business insights using Python

Part 1: Generate Food Delivery Dataset

nano generate_food_orders.py

```
import random
import datetime
```

```
foods = [
    "Pizza", "Burger", "Pasta", "Sandwich",
    "Biryani", "Noodles", "Fries", "Momos"
]
```

```
cities = [
    "Mumbai", "Delhi", "Bangalore",
    "Chennai", "Hyderabad", "Pune",
    "Ahmedabad", "Kolkata"
]
```

```

customers = [
    f"CUST{str(i).zfill(5)}"
    for i in range(1, 2001)
]

start_date = datetime.date(2025, 1, 1)
end_date = datetime.date(2025, 12, 31)

days = (end_date - start_date).days

print("Date,FoodItem,Price,City,CustomerID,Quantity")

for _ in range(60000):

    date = start_date + datetime.timedelta(
        days=random.randint(0, days)
    )

    food = random.choice(foods)

    price = round(
        random.uniform(80, 1200),
        2
    )

    city = random.choice(cities)

    customer = random.choice(customers)

    quantity = random.randint(1, 6)

    print(
        f"{date},{food},{price},{city},{customer},{quantity}"
    )

```

```
python3 generate_food_orders.py > food_orders.csv
```

```

ls -lh food_orders.csv
wc -l food_orders.csv
head -5 food_orders.csv

```

Part 2: Mapper - City Revenue

nano mapper_city_revenue.py

```
#!/usr/bin/env python3
```

```
import sys
```

```
next(sys.stdin)
```

```
for line in sys.stdin:
```

```
    parts = line.strip().split(",")
```

```
    if len(parts) != 6:  
        continue
```

```
    city = parts[3]
```

```
    try:  
        price = float(parts[2])  
        qty = int(parts[5])
```

```
        revenue = price * qty
```

```
        print(f"{city}\t{revenue}")
```

```
    except:  
        continue
```

Part 3: Reducer - City Revenue

nano reducer_city_revenue.py

```
#!/usr/bin/env python3
```

```
import sys
```

```
current_city = None  
total_revenue = 0  
orders = 0
```

```
for line in sys.stdin:
```

```

city, revenue = line.strip().split("\t")

revenue = float(revenue)

if city == current_city:

    total_revenue += revenue
    orders += 1

else:

    if current_city:

        print(
            f"{current_city}\t{total_revenue:.2f}\t{orders}"
        )

    current_city = city
    total_revenue = revenue
    orders = 1

if current_city:

    print(
        f"{current_city}\t{total_revenue:.2f}\t{orders}"
    )

```

Part 4: Local Testing

```

head -20 food_orders.csv \
| python3 mapper_city_revenue.py \
| sort \
| python3 reducer_city_revenue.py

```

Part 5: Upload To HDFS

```
hdfs dfs -mkdir -p /user/$USER/food/input
```

```
hdfs dfs -put food_orders.csv \
/user/$USER/food/input/
```

```
hdfs dfs -ls /user/$USER/food/input
```

Part 6: Run Hadoop Streaming

```
hadoop jar \  
$HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \  
-input /user/$USER/food/input/food_orders.csv \  
-output /user/$USER/food/output \  
-mapper "python3 mapper_city_revenue.py" \  
-reducer "python3 reducer_city_revenue.py" \  
-file mapper_city_revenue.py \  
-file reducer_city_revenue.py
```

Part 7: Download Results

```
hdfs dfs -get \  
/user/$USER/food/output/part-00000 \  
city_revenue.tx
```

```
cat city_revenue.txt
```

```
sort -t$t' -k2 -rn city_revenue.txt | head -5
```

Part 8: Monthly Revenue Analysis

```
nano mapper_monthly_revenue.py
```

```
#!/usr/bin/env python3
```

```
import sys
```

```
next(sys.stdin)
```

```
for line in sys.stdin:
```

```
    parts = line.strip().split(",")
```

```
    date = parts[0][:7]
```

```
    revenue = (  
        float(parts[2])  
        * int(parts[5])  
    )
```

```
    print(f"{date}\t{revenue}")
```

```
#!/usr/bin/env python3

import sys

current_month = None
total = 0

for line in sys.stdin:

    month, revenue = line.strip().split("\t")

    revenue = float(revenue)

    if month == current_month:

        total += revenue

    else:

        if current_month:

            print(
                f"{current_month}\t{total:.2f}"
            )

            current_month = month
            total = revenue

        if current_month:

            print(
                f"{current_month}\t{total:.2f}"
            )
```

Part 9: Food Item Quantity Analysis

```
#!/usr/bin/env python3

import sys

next(sys.stdin)

for line in sys.stdin:
```

```
parts = line.strip().split(",")
```

```
item = parts[1]
```

```
qty = int(parts[5])
```

```
print(f"{item}\t{qty}")
```

Screenshots

```
pratham@LAPTOP-PL4G895R:~/CustomerProject$ nano generate_food_orders.py
pratham@LAPTOP-PL4G895R:~/CustomerProject$ ls -l
total 8
-rw-r--r-- 1 pratham pratham 494 Jun 21 12:59 customers.csv
-rw-r--r-- 1 pratham pratham  0 Jun 21 13:11 food_orders.csv
-rw-r--r-- 1 pratham pratham 839 Jun 21 13:12 generate_food_orders.py
```

```
pratham@LAPTOP-PL4G895R:~/CustomerProject$ python3 generate_food_orders.py > food_orders.csv
pratham@LAPTOP-PL4G895R:~/CustomerProject$ ls -lh food_orders.csv
wc -l food_orders.csv
head -5 food_orders.csv
-rw-r--r-- 1 pratham pratham 2.6M Jun 21 13:13 food_orders.csv
60001 food_orders.csv
Date,FoodItem,Price,City,CustomerID,Quantity
2025-04-13,Pasta,983.07,Ahmedabad,CUST00261,6
2025-07-02,Momos,505.9,Ahmedabad,CUST00340,4
2025-12-08,Pizza,930.86,Bangalore,CUST01683,6
2025-09-29,Pasta,899.16,Kolkata,CUST01290,1
```

```
pratham@LAPTOP-PL4G895R:~/CustomerProject$ ls -l generate_food_orders.py
-rw-r--r-- 1 pratham pratham 839 Jun 21 13:12 generate_food_orders.py
```

```
pratham@LAPTOP-PL4G895R:~/CustomerProject$ ls -lh food_orders.csv
wc -l food_orders.csv
head -5 food_orders.csv
-rw-r--r-- 1 pratham pratham 2.6M Jun 21 13:13 food_orders.csv
60001 food_orders.csv
Date,FoodItem,Price,City,CustomerID,Quantity
2025-04-13,Pasta,983.07,Ahmedabad,CUST00261,6
2025-07-02,Momos,505.9,Ahmedabad,CUST00340,4
2025-12-08,Pizza,930.86,Bangalore,CUST01683,6
2025-09-29,Pasta,899.16,Kolkata,CUST01290,1
```

```
pratham@LAPTOP-PL4G895R:~/CustomerProject$ nano mapper_city_revenue.py
pratham@LAPTOP-PL4G895R:~/CustomerProject$ nano reducer_city_revenue.py
pratham@LAPTOP-PL4G895R:~/CustomerProject$ head -20 food_orders.csv \
| python3 mapper_city_revenue.py \
| sort \
| python3 reducer_city_revenue.py
Ahmedabad      10594.60      3
Bangalore      9770.78      3
Chennai 1709.32 1
Hyderabad      8196.48      4
Kolkata 11549.42 4
Mumbai  3236.05 1
Pune      6189.91      3
```

```
pratham@LAPTOP-PL4G895R:~/CustomerProject$ hdfs dfs -mkdir -p /user/$USER/food/input

hdfs dfs -put food_orders.csv \
/user/$USER/food/input/
```

```
pratham@LAPTOP-PL4G895R:~/CustomerProject$ hdfs dfs -ls /user/$USER/food/input
2026-06-21 13:15:05,913 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r--  1 pratham supergroup    2703541 2026-06-21 13:15 /user/pratham/food/input/food_orders.csv
```

```
pratham@LAPTOP-PL4G895R:~/CustomerProject$ hadoop jar \
$HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-input /user/$USER/food/input/food_orders.csv \
-output /user/$USER/food/output \
-mapper "python3 mapper_city_revenue.py" \
-reducer "python3 reducer_city_revenue.py" \
-file mapper_city_revenue.py \
-file reducer_city_revenue.py
```

```
pratham@LAPTOP-PL4G895R:~/CustomerProject$ cat city_revenue.txt
Ahmedabad      16917350.34      7517
Bangalore      16907557.66      7489
Chennai 16637451.65      7510
Delhi 16636112.77      7465
Hyderabad      16879807.72      7535
Kolkata 16757373.07      7482
Mumbai 16792788.77      7485
Pune 16728522.14      7517
```

```
pratham@LAPTOP-PL4G895R:~/CustomerProject$ sort -t$'\t' -k2 -rn city_revenue.txt | head -5
Ahmedabad      16917350.34      7517
Bangalore      16907557.66      7489
Hyderabad      16879807.72      7535
Mumbai 16792788.77      7485
Kolkata 16757373.07      7482
```

5. Conclusion

This project successfully demonstrates the use of Hadoop Streaming with Python MapReduce for processing large-scale food delivery data. A synthetic dataset containing thousands of food orders was generated and stored in HDFS. Using custom mapper and reducer programs, city-wise revenue, monthly sales trends, and product quantity analysis were performed efficiently. The results were further visualized using Python libraries such as Pandas and Matplotlib to derive meaningful business insights. Overall, the project highlights the practical application of Big Data Analytics, Hadoop Ecosystem, Python Programming, and Data Visualization for analyzing large datasets in a distributed environment.